

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
```

```
{
```

```
    int i= 1;
```

```
    int s =1;
```

```
    while(s <= n)
```

```
    {
```

```
        i++;
```

```
        s += i;
```

```
    }
```

```
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

Answer: (penalty regime: 0 %)

```

1  #include<stdio.h>
2  void function(int n){
3      int count=0;
4      count++;
5      int i=1;
6      count++;
7      int s=1;
8      count++;
9      while(s<=n){
10         count++;
11         i++;
12         count++;
13         s+=i;
14         count++;
15     }
16     printf("%d",count);
17 }
18 int main(){
19     int n;
20     scanf("%d",&n);
21     function(n);
22 }
23
24
25
```

	Input	Expected	Got	
✓	9	12	12	✓
✓	4	9	9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

118.185.187.137/moodle/mod/quiz/review.php?attempt=348686&cmid=3307

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
        printf("");
    }
    else
    {
        for(int i=1; i<=n; i++)
        {
            for(int j=1; j<=n; j++)
            {
                printf("");
                printf("");
                break;
            }
        }
    }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer: (penalty regime: 0 %)

```
1  #include<stdio.h>
2  void func(int n)
3  {
4  int count=0;
5  count++;
6  if(n==1)
7  {
8  //printf("");
9  }
10 else
11 {
12 for(int i=1; i<=n; i++)
13 {
14 count++;
15 count++;
16 for(int j=1; j<=n; j++)
17 {
18 count++;
19 //printf("");
20 //printf("");
21 count++;
22 break;
23 }
24 count++;
25 }
26 count++;
27 }
28 printf("%d",count);
29 }
30 int main(){
31 int n;
32 scanf("%d",&n);
33 func(n);
34 }
```

	Input	Expected	Got	
?	2	12	12	?
?	1000	5002	5002	?
?	143	717	717	?

Passed all tests! ?

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
    for (i = 1; i <= num; ++i)
    {
        if (num % i == 0)
        {
            printf("%d ", i);
        }
    }
}
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
1  #include<stdio.h>
2  void Factor(int num) {
3      int count=0;
4      for (int i = 1; i <= num; ++i){
5          count++;
6          count++;
7          if (num % i == 0){
8              count++;
9
10             //printf("%d ", i);
11         }
12     }
13     count++;
14     printf("%d",count);
15 }
16 int main(){
17     int num;
18     scanf("%d",&num);
19     Factor(num);
20 }
```

	Input	Expected	Got	
✓	12	31	31	✓
✓	25	54	54	✓
✓	4	12	12	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

118.185.187.137/moodle/mod/quiz/review.php?attempt=349282&cmid=3309

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time

complexity using counter method.

```

void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}

```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.**Input:**

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```

1  #include<stdio.h>
2  void function(int n)
3  {
4      int count=0;
5      count++;
6      int c= 0;
7      count++;
8      for(int i=n/2; i<n; i++){
9          count++;
10         count++;
11         for(int j=1; j<n; j = 2 * j){
12             count++;
13             count++;
14             for(int k=1; k<n; k = k * 2){
15                 count++;
16                 count++;
17                 c++;
18             }
19         }
20     }
21     printf("%d",count);
22 }
23
24 int main(){
25     int n;
26     scanf("%d",&n);
27     function(n);
28 }

```

	Input	Expected	Got	
✓	4	30	30	✓
✓	10	212	212	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

118.185.187.137/moodle/mod/quiz/review.php?attempt=349283&cmid=3310

Question 1 | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
    int rev = 0, remainder;
    while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;
    }
    print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```
1  #include<stdio.h>
2
3  void reverse(int n)
4  {
5      int count=0;
6      count++;
7      int rev = 0;
8      count++;
9      int remainder;
10     while (n != 0)
11     {
12         count++;
13         remainder = n % 10;
14         count++;
15         rev = rev * 10 + remainder;
16         count++;
17         n/= 10;
18         count++;
19     }
20     count++;
21     //print(rev);
22     printf("%d",count);
23 }
24 int main(){
25     int n;
26     scanf("%d",&n);
27     reverse(n);
28 }
```

	Input	Expected	Got	
✓	12	11	11	✓
✓	1234	19	19	✓

Passed all tests! ✓

Question 1 | Correct Mark 1.00 out of 1.00

Write a program to take value V and we want to make change for V Rs, and we have in finite supply of each of the denominations in Indian currency, i.e., we have in finite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

Answer: (penalty regime: 0 %)

```
1 #include<stdio.h>
2 int main(){
3     int v,count=0;
4     int denominations[]={1000,500,100,50,20,10,5,2,1};
5     scanf("%d",&v);
6     for(int i=0;i<9;i++){
7         count+=v/denominations[i];
8         v=v%denominations[i];
9     }
10    printf("%d",count);
11    return 0;
12 }
```

	Input	Expected	Got	
?	49	5	5	?

Passed all tests! ?

Correct

Marks for this submission: 1.00/1.00.

Question 1 | Correct Mark 1.00 out of 1.00

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor $g[i]$, which is the minimum size of a cookie that the child will be content with; and each cookie j has a size $s[j]$. If $s[j] \geq g[i]$, we can assign the cookie j to the child i , and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:**Input:**

```
3
1 2 3
2
1 1
```

Output:

```
1
```

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

$1 \leq g.length \leq 3 \times 10^4$

$0 \leq s.length \leq 3 \times 10^4$

$1 \leq g[i], s[j] \leq 2^{31} - 1$

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2
3 int main() {
4     int n, m;
5     scanf("%d", &n);
6
7     int g[n];
8     for (int i = 0; i < n; i++)
9         scanf("%d", &g[i]);
10
11     scanf("%d", &m);
12
13     int s[m];
14     for (int i = 0; i < m; i++)
15         scanf("%d", &s[i]);
16
17     for (int i = 0; i < n - 1; i++) {
18         for (int j = i + 1; j < n; j++) {
19             if (g[i] > g[j]) {
20                 int temp = g[i];
21                 g[i] = g[j];
22                 g[j] = temp;
23             }
24         }
25     }
26
27     for (int i = 0; i < m - 1; i++) {
28         for (int j = i + 1; j < m; j++) {
29             if (s[i] > s[j]) {
30                 int temp = s[i];
31                 s[i] = s[j];
32                 s[j] = temp;
33             }
34         }
35     }
36
37     int i = 0, j = 0, count = 0;
38
39     while (i < n && j < m) {
40         if (s[j] >= g[i]) {
41             count++;
42             i++;
43             j++;
44         } else {
45             j++;
46         }
47     }
```

```
47     }
48
49     printf("%d", count);
50
51     return 0;
52 }
```

	Input	Expected	Got	
?	2	2	2	?
	1 2			
	3			
	1 2 3			

Passed all tests! ?

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct | Mark 1.00 out of 1.00

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories.

If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3

burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$.

But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance

he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm. Apply greedy approach to solve the problem.

Input Format

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

```
3
5 10 7
```

Sample Output

```
76
```

For example:

Test	Input	Result
Test Case 1	3 1 3 2	18

Answer: (penalty regime: 0 %)

```

1  #include<stdio.h>
2  #include<math.h>
3  int main(){
4      int n;
5      scanf("%d",&n);
6      int arr[n];
7      for(int i=0;i<n;i++){
8          scanf("%d",&arr[i]);
9      }
10     for (int i = 0; i < n - 1; i++) {
11         for (int j = 0; j < n - i - 1; j++) {
12             if (arr[j] < arr[j + 1]) {
13                 int temp = arr[j];
14                 arr[j] = arr[j + 1];
15                 arr[j + 1] = temp;
16             }
17         }
18     }
19     int s=0;
20     for(int i=0;i<n;i++){
21         s+=(pow(n,i))*arr[i];
22     }
23     printf("%d",s);
24 }
25
```

	Test	Input	Expected	Got	
✓	Test Case 1	3 1 3 2	18	18	✓
✓	Test Case 2	4 7 4 9 6	389	389	✓
✓	Test Case 3	3 5 10 7	76	76	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

55 19

Question 1 | Correct | Mark 1.00 out of 1.00

Given an array of N integer, we have to maximize the sum of $arr[i] * i$, where i is the index of the element ($i = 0, 1, 2, \dots, N$). Write an algorithm based on Greedy technique with a Complexity $O(n \log n)$.

Input Format:

First line specifies the number of elements- n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int compare(const void *a, const void *b) {
5      return (*(int *)a - *(int *)b);
6  }
7
8  int main() {
9      int n;
10     scanf("%d", &n);
11
12     int a[n];
13
14     for (int i = 0; i < n; i++)
15         scanf("%d", &a[i]);
16
17     qsort(a, n, sizeof(int), compare);
18
19     long long sum = 0;
20
21     for (int i = 0; i < n; i++) {
22         sum += (long long)a[i] * i;
23     }
24
25     printf("%lld", sum);
26
27     return 0;
28 }
```

	Input	Expected	Got	
✓	5	40	40	✓
	2			
	5			
	3			
	4			
	0			

	Input	Expected	Got	
✓	10 2 2 2 4 4 3 3 5 5 5	191	191	✓
✓	2 45 3	45	45	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct | Mark 1.00 out of 1.00

Given two arrays `array_One[]` and `array_Two[]` of same size `N`. We need to first rearrange the arrays such that the sum of the product of pairs (1 element from each) is minimum. That is $\text{SUM}(A[i] * B[i])$ for all `i` is minimum.

For example:

Input	Result
3	28
1	
2	
3	
4	
5	
6	

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int main() {
4      int n, i, j, temp;
5      scanf("%d", &n);
6
7      int a[n], b[n];
8
9      for (i = 0; i < n; i++)
10         scanf("%d", &a[i]);
11
12     for (i = 0; i < n; i++)
13         scanf("%d", &b[i]);
14
15     for (i = 0; i < n - 1; i++) {
16         for (j = i + 1; j < n; j++) {
17             if (a[i] > a[j]) {
18                 temp = a[i];
19                 a[i] = a[j];
20                 a[j] = temp;
21             }
22         }
23     }
24
25     for (i = 0; i < n - 1; i++) {
26         for (j = i + 1; j < n; j++) {
27             if (b[i] < b[j]) {
28                 temp = b[i];
29                 b[i] = b[j];
30                 b[j] = temp;
31             }
32         }
33     }
34
35     long long sum = 0;
36
37     for (i = 0; i < n; i++)
38         sum += (long long)a[i] * b[i];
39
40     printf("%lld", sum);
41
42     return 0;
43 }
```

	Input	Expected	Got	
✓	3 1 2 3 4 5 6	28	28	✓
✓	4 7 5 1 2 1 3 4 1	22	22	✓
✓	5 20 10 30 10 40 8 9 4 3 10	590	590	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Back to Course

Question 1 | Correct | Mark 1.00 out of 1.00**Problem Statement**

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2
3 int findFirstZero(int a[], int low, int high) {
4     if (low > high)
5         return -1;
6
7     int mid = (low + high) / 2;
8
9     if (a[mid] == 0) {
10         if (mid == 0 || a[mid - 1] == 1)
11             return mid;
12
13         return findFirstZero(a, low, mid - 1);
14     }
15
16     return findFirstZero(a, mid + 1, high);
17 }
18
19 int main() {
20     int n;
21     scanf("%d", &n);
22
23     int a[n];
24
25     for (int i = 0; i < n; i++)
26         scanf("%d", &a[i]);
27
28     int index = findFirstZero(a, 0, n - 1);
29
30     if (index == -1)
31         printf("0");
32     else
33         printf("%d", n - index);
34
35     return 0;
36 }

```

	Input	Expected	Got	
✓	5 1 1 1 0 0	2	2	✓

	Input	Expected	Got	
?	10 1 1 1 1 1 1 1 1 1 1 1 1	0	0	?
?	8 0 0 0 0 0 0 0 0 0	8	8	?
?	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	?

Passed all tests! ?

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-231 <= nums[i] <= 231 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int main() {
4      int n;
5      scanf("%d", &n);
6
7      int nums[n];
8
9      for (int i = 0; i < n; i++)
10         scanf("%d", &nums[i]);
11
12     int candidate = nums[0];
13     int count = 1;
14
15     for (int i = 1; i < n; i++) {
16         if (nums[i] == candidate)
17             count++;
18         else
19             count--;
20
21         if (count == 0) {
22             candidate = nums[i];
23             count = 1;
24         }
25     }
26
27     printf("%d", candidate);
28
29     return 0;
30 }
```

	Input	Expected	Got	
?	3	3	3	?
	3 2 3			

Passed all tests! ?

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

118.185.187.137/moodle/mod/quiz/review.php?attempt=363320&cmid=3313

Question 1 | Correct | Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int floorValue(int a[], int low, int high, int x) {
4      if (low > high)
5          return -1;
6
7      int mid = (low + high) / 2;
8
9      if (a[mid] == x)
10         return a[mid];
11
12     if (a[mid] < x) {
13         int result = floorValue(a, mid + 1, high, x);
14
15         if (result == -1)
16             return a[mid];
17         else
18             return result;
19     }
20
21     return floorValue(a, low, mid - 1, x);
22 }
23
24 int main() {
25     int n, x;
26     scanf("%d", &n);
27
28     int a[n];
29
30     for (int i = 0; i < n; i++)
31         scanf("%d", &a[i]);
32
33     scanf("%d", &x);
34
35     int result = floorValue(a, 0, n - 1, x);
36
37     printf("%d", result);
38
39     return 0;
40 }
```

	Input	Expected	Got	
?	6 1 2 8 10 12 19 5	2	2	?
?	5 10 22 85 108 129 100	85	85	?
?	7 3 5 7 9 11 13 15 10	9	9	?

Passed all tests! ?

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct | Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int findPair(int a[], int left, int right, int x) {
4      if (left >= right)
5          return 0;
6
7      int sum = a[left] + a[right];
8
9      if (sum == x) {
10         printf("%d\n", a[left]);
11         printf("%d\n", a[right]);
12         return 1;
13     }
14
15     if (sum < x)
16         return findPair(a, left + 1, right, x);
17     else
18         return findPair(a, left, right - 1, x);
19 }
20
21 int main() {
22     int n, x;
23
24     scanf("%d", &n);
25
26     int a[n];
27
28     for (int i = 0; i < n; i++)
29         scanf("%d", &a[i]);
30
31     scanf("%d", &x);
32
33     if (!findPair(a, 0, n - 1, x))
34         printf("No");
35
36     return 0;
37 }

```

	Input	Expected	Got	
✓	4	4	4	✓
	2	10	10	
	4			
	8			
	10			
	14			

	Input	Expected	Got	
?	5	No	No	?
	2			
	4			
	6			
	8			
	10			
	100			

Passed all tests! ?

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct | Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5	12 34 67 78 98
67 34 12 98 78	

Answer:

```

1  #include <stdio.h>
2
3  void quickSort(int a[], int low, int high) {
4      if (low < high) {
5          int i = low;
6          int j = high;
7          int pivot = a[low];
8
9          while (i < j) {
10             while (a[i] <= pivot && i < high)
11                 i++;
12
13             while (a[j] > pivot)
14                 j--;
15
16             if (i < j) {
17                 int temp = a[i];
18                 a[i] = a[j];
19                 a[j] = temp;
20             }
21         }
22
23         // Place pivot in correct position
24         int temp = a[low];
25         a[low] = a[j];
26         a[j] = temp;
27
28         quickSort(a, low, j - 1);
29         quickSort(a, j + 1, high);
30     }
31 }
32
33 int main() {
34     int n;
35     scanf("%d", &n);
36
37     int a[n];
38
39     for (int i = 0; i < n; i++)
40         scanf("%d", &a[i]);
41
42     quickSort(a, 0, n - 1);
43
44     for (int i = 0; i < n; i++)
45         printf("%d ", a[i]);
46
47     return 0;
48 }

```

	Input	Expected	Got	
	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	
	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	
	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	

Passed all tests! 

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct Mark 10.00 out of 10.00**Playing with Numbers:**

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram's turn, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3. Write any efficient algorithm to find the possible ways.

Example 1:**Input:** 6**Output:** 6**Explanation:** There are 6 ways to represent number with 1 and 3

1+1+1+1+1+1

3+3

1+1+1+3

1+1+3+1

1+3+1+1

3+1+1+1

Input Format

First Line contains the number n

Output Format**Print: The number of possible ways 'n' can be represented using 1 and 3****Sample Input**

6

Sample Output

6

Answer: (penalty regime: 0%)

Ace editor not ready. Perhaps reload page?

Falling back to raw text area.

```
#include <stdio.h>
#include <stdlib.h>

long long find_ways(int n) {
    if (n < 0) return 0;
    if (n == 0 || n == 1 || n == 2) return 1;

    long long *dp = (long long *)malloc((n + 1) * sizeof(long long));
    if (dp == NULL) {
        return -1;
    }

    dp[0] = 1;
    dp[1] = 1;
    dp[2] = 1;

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 3];
    }
}
```

	Input	Expected	Got	
	6	6	6	
	25	8641	8641	
	100	24382819596721629	24382819596721629	

Passed all tests! 

Correct

Marks for this submission: 10.00/10.00.

[Back to Course](#)

Question 1 | Correct | Mark 10.00 out of 10.00**Playing with Chessboard:**

Ram is given with an $n \times n$ chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:**Input**

```
3
1 2 4
2 3 4
8 7 1
```

Output:

```
19
```

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value: $1+2+8+7+1=19$

Input Format

First Line contains the integer n

The next n lines contain the $n \times n$ chessboard values

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main() {
5     int n;
6     if (scanf("%d", &n) != 1) return 0;
7
8     long long **board = (long long **)malloc(n * sizeof(long long *));
9     long long **dp = (long long **)malloc(n * sizeof(long long *));
10    for (int i = 0; i < n; i++) {
11        board[i] = (long long *)malloc(n * sizeof(long long));
12        dp[i] = (long long *)malloc(n * sizeof(long long));
13    }
14
15    for (int i = 0; i < n; i++) {
16        for (int j = 0; j < n; j++) {
17            scanf("%lld", &board[i][j]);
18        }
19    }
20
21    dp[0][0] = board[0][0];
22
23    for (int j = 1; j < n; j++) {
24        dp[0][j] = dp[0][j - 1] + board[0][j];
25    }
26
27    for (int i = 1; i < n; i++) {
28        dp[i][0] = dp[i - 1][0] + board[i][0];
29    }
30
31    for (int i = 1; i < n; i++) {
32        for (int j = 1; j < n; j++) {
33            long long max_prev = (dp[i - 1][j] > dp[i][j - 1]) ? dp[i - 1][j] : dp[i][j - 1];
34            dp[i][j] = board[i][j] + max_prev;
35        }
36    }
```

```

37
38     printf("%lld\n", dp[n - 1][n - 1]);
39
40     for (int i = 0; i < n; i++) {
41         free(board[i]);
42         free(dp[i]);
43     }
44     free(board);
45     free(dp);
46
47     return 0;
48 }

```

	Input	Expected	Got	
✓	3 1 2 4 2 3 4 8 7 1	19	19	✓
✓	3 1 3 1 1 5 1 4 2 1	12	12	✓
✓	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✓

Passed all tests! ✓

Correct

Marks for this submission: 10.00/10.00.

[Back to Course](#)



Question 1 | Correct | Mark 1.00 out of 1.00

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1		a		g		g		t		a		b
s2		g		x		t		x		a		y
										b		

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab	2
azb	

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  int max(int a, int b) {
6      return (a > b) ? a : b;
7  }
8
9  int longestCommonSubsequence(char* s1, char* s2) {
10     int m = strlen(s1);
11     int n = strlen(s2);
12
13     int **dp = (int **)malloc((m + 1) * sizeof(int *));
14     for (int i = 0; i <= m; i++) {
15         dp[i] = (int *)malloc((n + 1) * sizeof(int));
16     }
17
18     for (int i = 0; i <= m; i++) {
19         for (int j = 0; j <= n; j++) {
20             if (i == 0 || j == 0) {
21                 dp[i][j] = 0;
22             }
23             else if (s1[i - 1] == s2[j - 1]) {
24                 dp[i][j] = dp[i - 1][j - 1] + 1;
25             }
26             else {
27                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
28             }
29         }
30     }
31
32     int result = dp[m][n];
33
34     for (int i = 0; i <= m; i++) {
35         free(dp[i]);
36     }
37     free(dp);
38
39     return result;
40 }
41
42 int main() {
43     char s1[1000], s2[1000];

```

```
44 |
45 | if (scanf("%s %s", s1, s2) == 2) {
46 |     int lcs_length = longestCommonSubsequence(s1, s2);
47 |     printf("%d\n", lcs_length);
48 | }
49 |
50 | return 0;
51 | }
```

	Input	Expected	Got	
✓	aab azb	2	2	✓
✓	ABCD ABCD	4	4	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct | Mark 1.00 out of 1.00

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence: [-1,3,4,5,2,2,2,3]

the subsequence is [-1,2,2,2,3]

Output:6

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int longestNonDecreasingSubsequence(int* arr, int n) {
5      if (n == 0) return 0;
6
7      int *dp = (int *)malloc(n * sizeof(int));
8      int max_len = 1;
9
10     for (int i = 0; i < n; i++) {
11         dp[i] = 1;
12     }
13
14     for (int i = 1; i < n; i++) {
15         for (int j = 0; j < i; j++) {
16             if (arr[i] >= arr[j] && dp[j] + 1 > dp[i]) {
17                 dp[i] = dp[j] + 1;
18             }
19         }
20         if (dp[i] > max_len) {
21             max_len = dp[i];
22         }
23     }
24
25     free(dp);
26     return max_len;
27 }
28
29 int main() {
30     int n;
31     if (scanf("%d", &n) != 1) return 0;
32
33     int *arr = (int *)malloc(n * sizeof(int));
34     for (int i = 0; i < n; i++) {
35         scanf("%d", &arr[i]);
36     }
37
38     int result = longestNonDecreasingSubsequence(arr, n);
39     printf("%d\n", result);
40
41     free(arr);
42     return 0;
43 }
```

	Input	Expected	Got	
✓	9 -1 3 4 5 2 2 2 3	6	6	✓
✓	7 1 2 2 4 5 7 6	6	6	✓

10/5/26, 3:28 PM

4-DP-Longest non-decreasing Subsequence: Attempt review

Passed all tests! 

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

118.185.187.137/moodle/mod/quiz/review.php?attempt=368180&cmid=3325

Question 1 | Correct | Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n , find one number that repeats.

Input Format:

First Line - Number of elements

 n Lines - n Elements

Output Format:

Element x - That is repeated**For example:**

Input	Result
5 1 1 2 3 4	1

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int findDuplicate(const int* nums, int n) {
5      int low = 1, high = n;
6
7      while (low < high) {
8          int mid = low + (high - low) / 2;
9          int count = 0;
10         for (int i = 0; i < n; i++) {
11             if (nums[i] <= mid) {
12                 count++;
13             }
14         }
15         if (count > mid) {
16             high = mid;
17         } else {
18             low = mid + 1;
19         }
20     }
21
22     return low;
23 }
24
25 int main() {
26     int n;
27     if (scanf("%d", &n) != 1) return 0;
28
29     int *nums = (int *)malloc(n * sizeof(int));
30     for (int i = 0; i < n; i++) {
31         scanf("%d", &nums[i]);
32     }
33
34     int duplicate = findDuplicate(nums, n);
35     printf("%d\n", duplicate);
36
37     free(nums);
38     return 0;
39 }

```

	Input	Expected	Got	
✓	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✓

	Input	Expected	Got	
	5 1 2 3 4 4	4	4	
	5 1 1 2 3 4	1	1	

Passed all tests! 

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

118.185.187.137/moodle/mod/quiz/review.php?attempt=368100&cmid=3326

Question 1 | Correct | Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int findDuplicate(const int* nums, int n) {
5      int slow = nums[0];
6      int fast = nums[0];
7
8      do {
9          slow = nums[slow];
10         fast = nums[nums[fast]];
11     } while (slow != fast);
12
13     slow = nums[0];
14     while (slow != fast) {
15         slow = nums[slow];
16         fast = nums[fast];
17     }
18
19     return slow;
20 }
21
22 int main() {
23     int n;
24     if (scanf("%d", &n) != 1) return 0;
25
26     int *nums = (int *)malloc(n * sizeof(int));
27     for (int i = 0; i < n; i++) {
28         scanf("%d", &nums[i]);
29     }
30
31     int duplicate = findDuplicate(nums, n);
32     printf("%d\n", duplicate);
33
34     free(nums);
35     return 0;
36 }

```

	Input	Expected	Got	
✓	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✓
✓	5 1 2 3 4 4	4	4	✓

	Input	Expected	Got	
?	5 1 1 2 3 4	1	1	?

Passed all tests! ?

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct | Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void solveIntersection() {
5     int n1;
6     if (scanf("%d", &n1) != 1) return;
7
8     int *arr1 = (int *)malloc(n1 * sizeof(int));
9     for (int i = 0; i < n1; i++) {
10         scanf("%d", &arr1[i]);
11     }
12
13     int n2;
14     scanf("%d", &n2);
15
16     int *arr2 = (int *)malloc(n2 * sizeof(int));
17     for (int i = 0; i < n2; i++) {
18         scanf("%d", &arr2[i]);
19     }
20
21     int i = 0, j = 0;
22     int first_element = 1;
23
24     while (i < n1 && j < n2) {
25         if (arr1[i] == arr2[j]) {
26             if (!first_element) {
27                 printf(" ");
28             }
29             printf("%d", arr1[i]);

```

```
30     first_element = 0;
31     i++;
32     j++;
33 } else if (arr1[i] < arr2[j]) {
34     i++;
35 } else {
36     j++;
37 }
38 }
39 printf("\n");
40
41 free(arr1);
42 free(arr2);
43 }
44
45 int main() {
46     int t;
47     if (scanf("%d", &t) != 1) return 0;
48
49     while (t--) {
50         solveIntersection();
51     }
52 }
```

	Input	Expected	Got	
✓	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✓
✓	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void solveIntersection() {
5     int n1;
6     if (scanf("%d", &n1) != 1) return;
7
8     int *arr1 = (int *)malloc(n1 * sizeof(int));
9     for (int i = 0; i < n1; i++) {
10         scanf("%d", &arr1[i]);
11     }
12
13     int n2;
14     scanf("%d", &n2);
15
16     int *arr2 = (int *)malloc(n2 * sizeof(int));
17     for (int i = 0; i < n2; i++) {
18         scanf("%d", &arr2[i]);
19     }
20
21     int i = 0, j = 0;
22     int first_element = 1;
23
24     while (i < n1 && j < n2) {
25         if (arr1[i] == arr2[j]) {
26             if (!first_element) {
27                 printf(" ");
28             }
29             printf("%d", arr1[i]);

```

```
30     first_element = 0;
31     i++;
32     j++;
33 } else if (arr1[i] < arr2[j]) {
34     i++;
35 } else {
36     j++;
37 }
38 }
39 printf("\n");
40
41 free(arr1);
42 free(arr2);
43 }
44
45 int main() {
46     int t;
47     if (scanf("%d", &t) != 1) return 0;
48
49     while (t--) {
50         solveIntersection();
51     }
52 }
```

	Input	Expected	Got	
✓	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✓
✓	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct | Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that $A[j] - A[i] = k$, $i \neq j$.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as $5 - 1 = 4$

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int hasPairWithDifference(const int* A, int n, int k) {
5      if (n < 2) return 0;
6
7      if (k == 0) {
8          for (int i = 0; i < n - 1; i++) {
9              if (A[i] == A[i + 1]) return 1;
10             }
11             return 0;
12         }
13
14         int i = 0, j = 1;
15         while (i < n && j < n) {
16             int diff = A[j] - A[i];
17             if (i != j && diff == k) {
18                 return 1;
19             } else if (diff < k) {
20                 j++;
21             } else {
22                 i++;
23                 if (i == j) {
24                     j++;
25                 }
26             }
27         }
28         return 0;
29     }
30
31     int main() {
32         int n;
33         if (scanf("%d", &n) != 1) return 0;
34
35         int *A = (int *)malloc(n * sizeof(int));
36         for (int i = 0; i < n; i++) {
37             scanf("%d", &A[i]);
38         }
39

```

```

40 |     int k;
41 |     scanf("%d", &k);
42 |
43 |     int result = hasPairWithDifference(A, n, k);
44 |     printf("%d\n", result);
45 |
46 |     free(A);
47 |     return 0;
48 | }

```

	Input	Expected	Got	
✓	3 1 3 5 4	1	1	✓
✓	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✓
✓	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✓
✓	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)

Question 1 | Correct | Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that $A[j] - A[i] = k$, $i \neq j$.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as $5 - 1 = 4$

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int hasPairWithDifference(const int* A, int n, int k) {
5      if (n < 2) return 0;
6
7      if (k < 0) k = -k;
8
9      int i = 0, j = 1;
10     while (i < n && j < n) {
11         if (i == j) {
12             j++;
13             continue;
14         }
15
16         int diff = A[j] - A[i];
17
18         if (diff == k) {
19             return 1;
20         } else if (diff < k) {
21             j++;
22         } else {
23             i++;
24         }
25     }
26
27     return 0;
28 }
29
30 int main() {
31     int n;
32     if (scanf("%d", &n) != 1) return 0;
33
34     int *A = (int *)malloc(n * sizeof(int));
35     for (int i = 0; i < n; i++) {
36         scanf("%d", &A[i]);
37     }
38
39     int k;

```

```

40     scanf("%d", &k);
41
42     int result = hasPairWithDifference(A, n, k);
43     printf("%d\n", result);
44
45     free(A);
46     return 0;
47 }

```

	Input	Expected	Got	
✓	3 1 3 5 4	1	1	✓
✓	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✓
✓	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✓
✓	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)